KLAi

Text as data and meaning as numbers

How can a computer compare customer messages when it only understands numbers?

AI and Data Science Professional | Session 21 of 24 | Module E: Neural networks and text
KLAi Professional Learning | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Clean text and turn it into numbers with a bag of words and TF-IDF, and classify sentiment

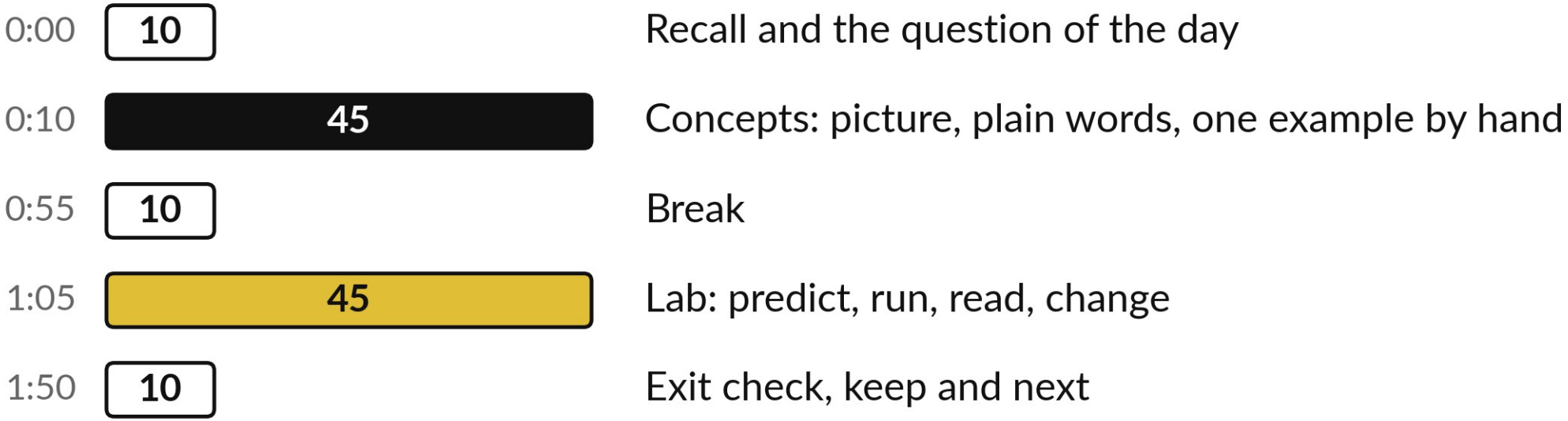


Score a search against an answer key, and explain why counting words misses negation and synonyms



Turn messages into embeddings and rank them by cosine similarity (semantic search)

Your two hours: concepts first, then a Python lab on customer messages



Pictures and plain words first. Then you open the notebook. Then the checks and what you keep.

Three questions from last time

1. What does one artificial neuron compute?

2. What does training a neural network do?

3. When do you keep a simpler model instead of a neural network?



Answer aloud first. Then we check together.

Answers to the three questions from Session 20

Question	Short answer
What does one artificial neuron compute?	A weighted sum of its inputs plus a bias, passed through an activation function such as sigmoid or ReLU.
What does training a neural network do?	It repeats small corrections, by gradient descent with backpropagation, to lower the loss function.
When do you keep a simpler model instead of a neural network?	When the simpler model scores about the same on the test set. It is cheaper and easier to explain (parsimony).

Words we use today

	Word	What it means
	**Text preprocessing (tokenisation, stop words)**	Cleaning raw text into tokens: lowercase it, remove noise with a regular expression, split it into words and drop filler words.
	**Bag of words and TF-IDF**	Bag of words counts each word in each message and ignores the order. TF-IDF weights those counts so rare words count more.
	**Text classification (sentiment analysis)**	A supervised model that gives each message a label, such as positive, neutral or negative.
	**Embedding (vector)**	A list of numbers that stands for the meaning of a message. Messages with similar meanings get similar vectors.
	**Cosine similarity and semantic search**	A score from -1 to 1 for how alike two vectors are. Semantic search ranks messages by that score.

Text is unstructured data: a sentence has no fixed columns to count

feedback.csv: each record is one customer message

rating	sentiment	theme	message
3	negative	price	Feedback: Too costly for ordinary families. Will recommend.
5	positive	loyalty	Feedback: Easy to earn points at Fort Portal. Keep it up.
1	negative	queue	The checkout line at Fort Portal was very slow today.
3	neutral	question	On Friday evening. Is diapers available in Jinja? Thanks.

The message is a text variable: a string with no fixed columns

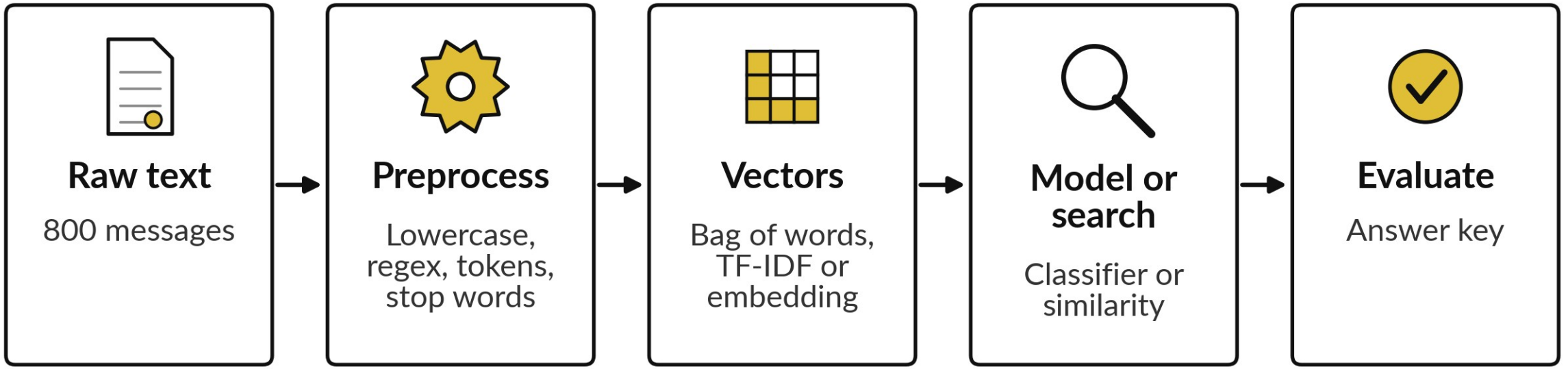
Unstructured data has no fixed columns. Free text is the most common kind.

feedback.csv holds 800 customer messages. Variable type: text (string).

A model needs a vector: an ordered list of numbers.

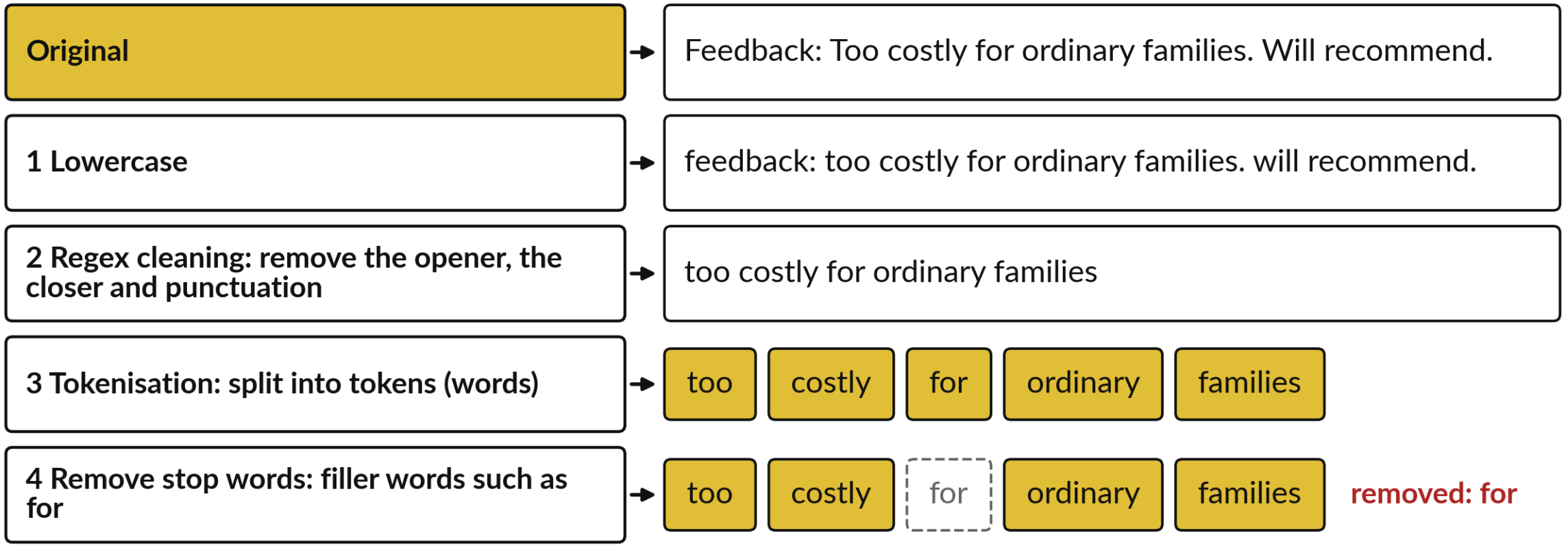
So we turn each message into a vector.

Every text method follows the same path: clean, make vectors, then classify or search



Today we walk this path twice: with counted words, and with embeddings.

Text preprocessing changes one real message in four steps: lowercase, regex, tokenise, drop stop words



Message F0001 of feedback.csv. Each step feeds the next.

A regular expression (regex) is a pattern that finds text, and one line cleans 800 messages

Pattern	Meaning	Example
`[^a-z\s]`	Any character that is not a letter or a space	Removes full stops and commas
`\bnot\b`	The whole word not	Found in 88 of 800 raw messages
`\d{3}`	Exactly three digits in a row	Part of a phone number
`^hello\.`	The text hello. at the start of a message	A greeting to strip

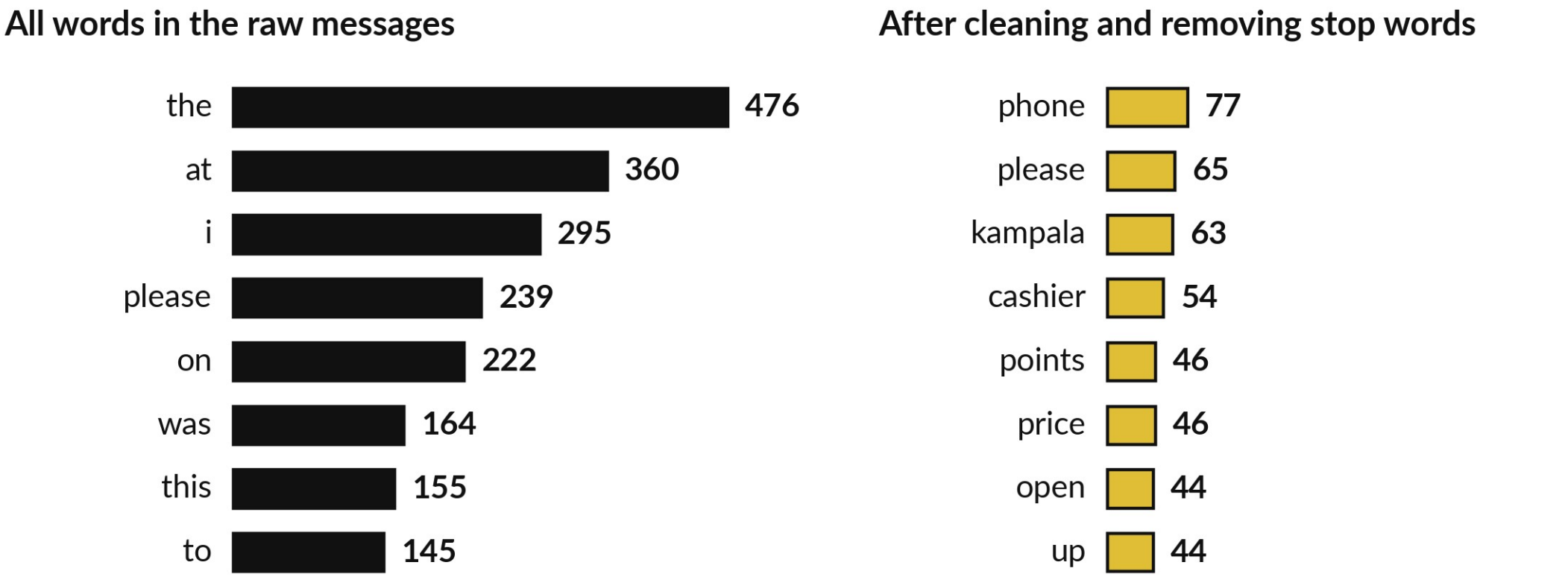
A regex is Find and Replace with wildcards. Python applies it with re.sub or str.replace.

Messages here open with Hello. or Feedback: and close with Please reply.

After we strip those lines, \bnot\b matches 35 messages, down from 88.

Test a regex on a few rows first.

The most common words are filler, and stop word removal exposes the words that carry meaning



Messages containing each word, out of 800. Keep the word not: many stop word lists remove it.

A bag of words counts each word in each message and ignores the order

A: The card machine at Jinja was not working so I could not pay.

B: The checkout line at Jinja was very slow today.

C: Fast checkout at Jinja, the cashier was quick.

Message	card	machine	jinja	not	working	so	pay	checkout	line	slow	today	fast	cashier	quick
Message A	1	1	1	2	1	1	1	0	0	0	0	0	0	0
Message B	0	0	1	0	0	0	0	1	1	1	1	0	0	0
Message C	0	0	1	0	0	0	0	1	0	0	0	1	1	1

Gold cell: the word appears. The number is how many times. Order is lost.

A document-term matrix (count vectorizer). The full file is 800 messages by 267 words.

TF-IDF multiplies a word count by how rare the word is across all messages

TF-IDF

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

Inverse document frequency

$$\text{idf}(t) = \ln\left(N/\text{df}(t)\right)$$

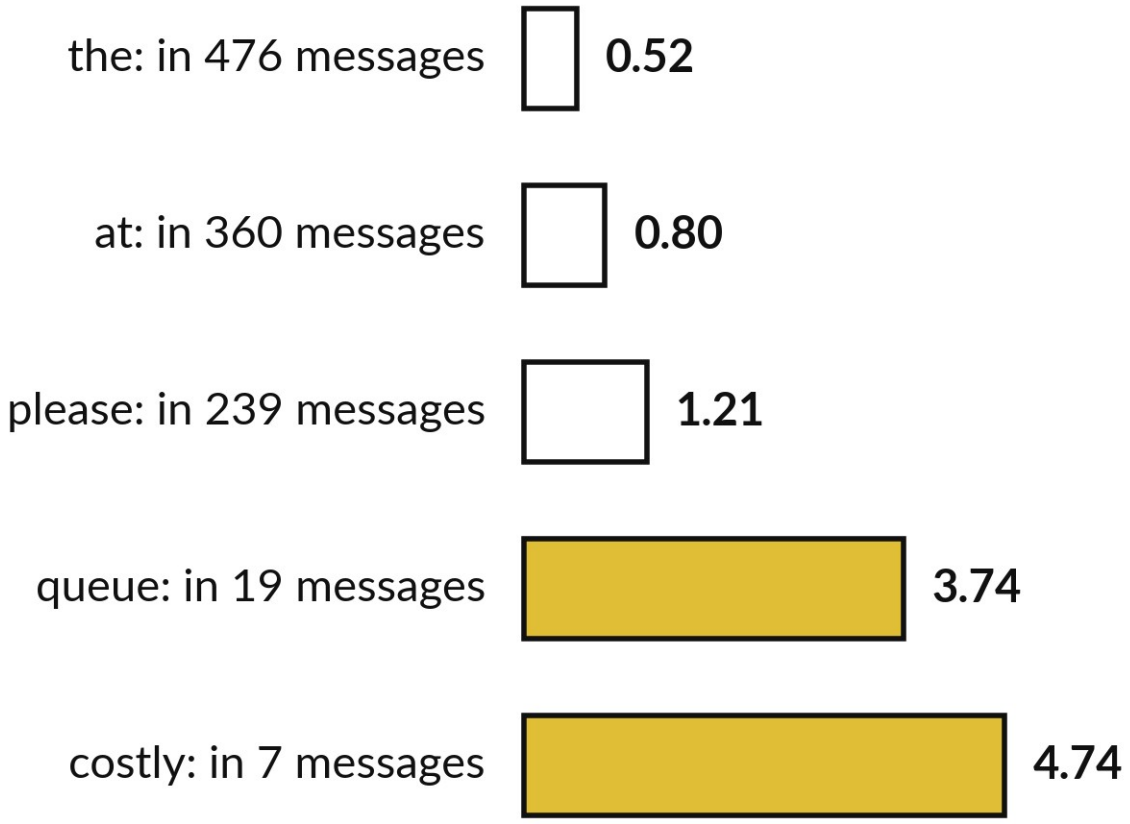
Our word: queue

$$\ln(800/19) = 3.74$$

Symbol	What it means	In our data
tf(t, d)	Term frequency: times word t appears in message d	queue: once
df(t)	Document frequency: messages that contain word t	queue: 19 messages
N	The number of messages (documents)	800
ln	Natural logarithm. A rarer word gives a larger value	ln(800/19) = 3.74

scikit-learn adds 1 inside the logarithm and 1 outside, so its numbers differ from ours.

Rare words get more weight: idf is 0.52 for the and 4.74 for costly



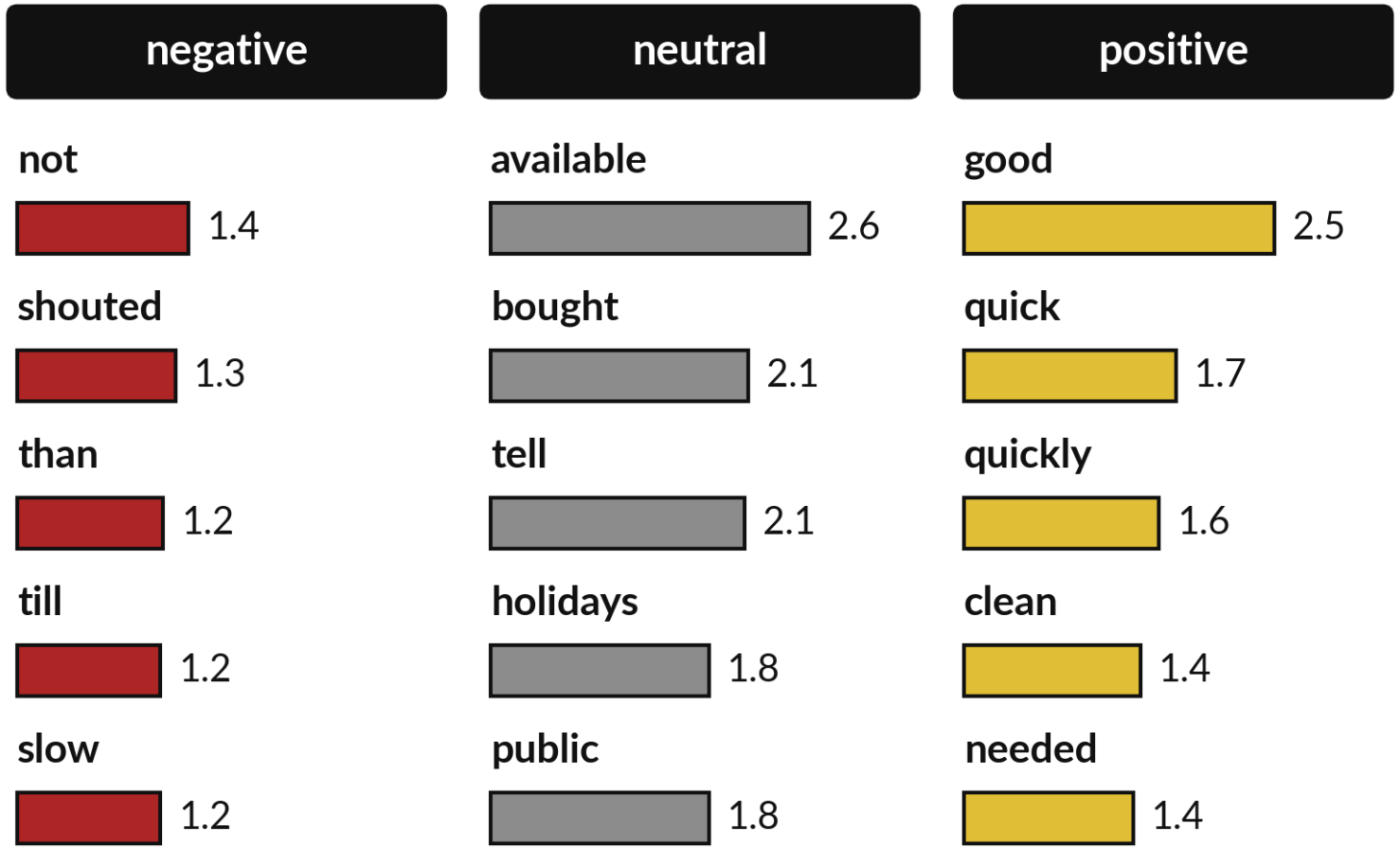
Inverse document frequency (idf) shrinks words found in many messages and raises words found in few.

Computed on all 800 raw messages, before stop words are removed.

TF-IDF down-weights filler even when you keep it.

Then each message vector is scaled to length 1.

A sentiment classifier learns one weight per word for each label



Text classification: a supervised model learns from messages with known labels.

Target: sentiment (negative, neutral, positive).

Features: the TF-IDF value of each word.

The chart shows the five largest learned weights for each label.

A random split flatters the score: 565 of 800 test messages repeat a training message

Shuffled 5-fold: 94.2%

565 of 800 test messages repeat a training text.

Data leakage: the score is too high.

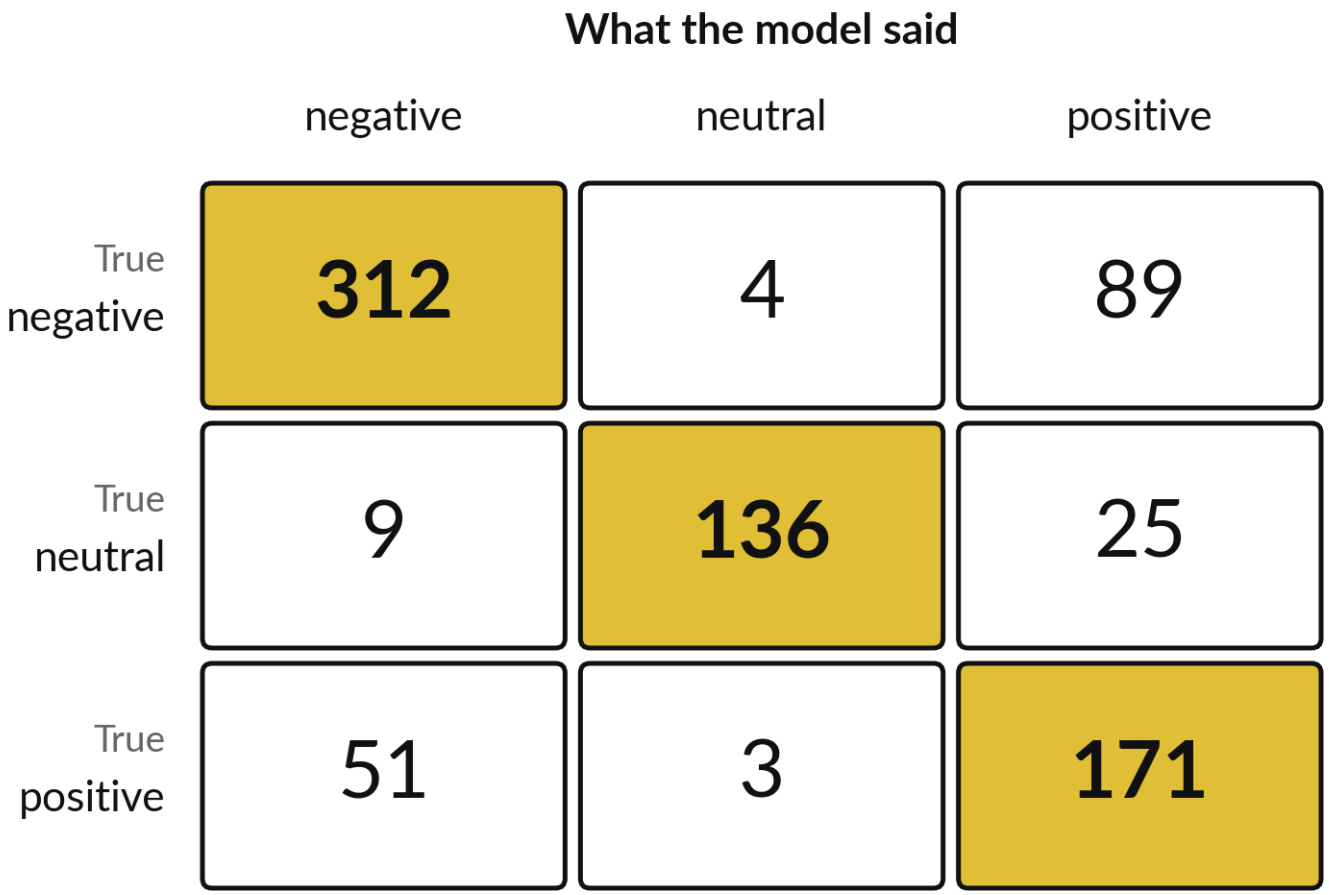
Grouped 5-fold: 77.4%

Copies of one text share a fold.

Only 360 different texts behind 800 messages.

Use the grouped score. The baseline is 50.6%.

Grouped cross-validation gives 77.4% accuracy against 50.6% for the baseline



619 of 800 messages are right when the model has never seen their text.

Five folds of 160 messages, each predicted by a model trained on the other four.

Fold scores run from 64% to 86%.

Baseline (always negative): 50.6%.

Read the mistakes before you trust the score: some are bad labels and some are model errors

	Message	Label	Model	Our reading
✗	**Mobile money payment went through quickly, thank you.**	positive	negative	Model wrong: the message is positive, and its words never appeared in the other folds.
✗	**Delivery was on time and the groceries were cold. Thank you.**	positive	negative	Arguable: the message is mixed.
✗	**Feedback: I was charged twice for one purchase. Thank you.**	positive	negative	Label looks wrong: the message is a complaint.
✗	**Staff were rude when I asked about a refund for bread.**	neutral	negative	Label looks wrong: the message is a complaint.

4 of the 181 mistakes. Real messages vary more than these 360 texts, so expect lower scores at work.

Counting words cannot read negation: our model still calls "not good" positive

The queue was good

Model says: positive.

Probability of positive: 0.94.

Correct.

The queue was not good

Model says: positive.

Probability of positive: 0.72.

Wrong: the word good still outweighs the word not.

A bag of words sees words and ignores their order. Models that read word order do better.

In practice you use the scikit-learn recipe: TfidfVectorizer, LogisticRegression and a pipeline

```
pipe = make_pipeline(
    TfidfVectorizer(),
    LogisticRegression(max_iter=1000))
pred = cross_val_predict(
    pipe, texts, y, groups=groups,
    cv=GroupKFold(5))
print(classification_report(y, pred))
```

```
Expected accuracy: about 0.79
(folds 0.71 to 0.88; numpy replica)
```

In plain words

CountVectorizer builds the bag of words. TfidfVectorizer builds TF-IDF.

make_pipeline chains the vectoriser and the classifier, so the vocabulary and idf come from the training folds only.

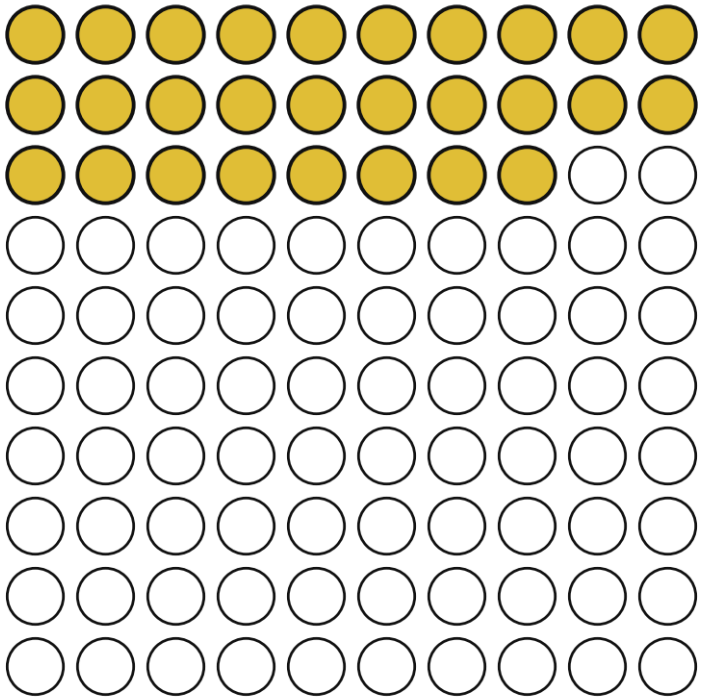
GroupKFold keeps copies of one text together.

scikit-learn idf for queue is 4.69, not 3.74, because of its smoothing.

A keyword search for queue finds only 28% of the queue messages

28%

We search the 360 different texts, so copies cannot inflate the result. The word queue is in 10 of the 36 texts whose theme is queue (recall). All 10 hits are right (precision 100 percent).



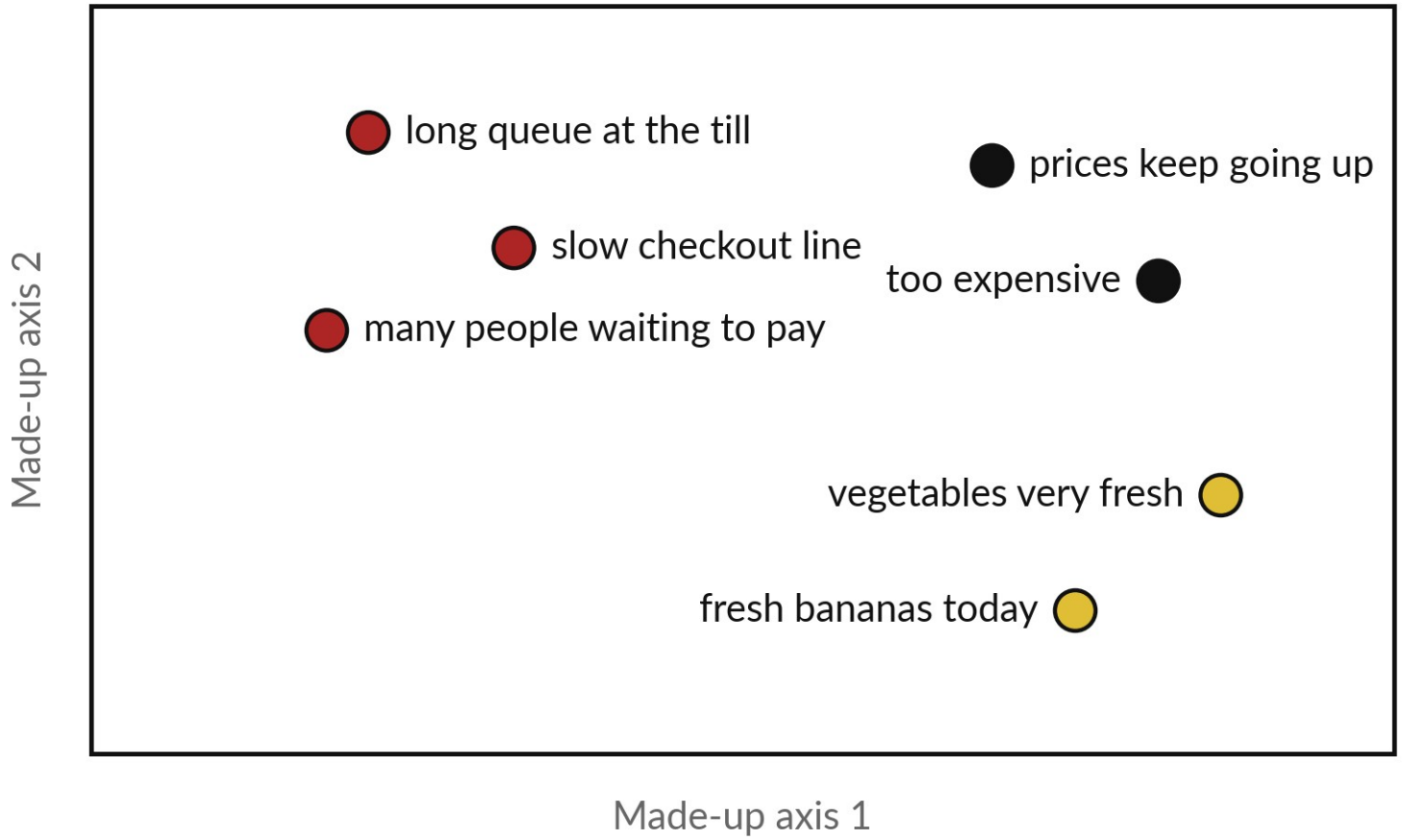
queue messages found in every 100

Adding keywords raises recall but lowers precision, and you must guess every synonym

	Keywords searched	Texts found	Right ones	Precision	Recall
🔍	**queue**	10	10	100%	28%
🔍	**queue, line**	19	19	100%	53%
🔍	**queue, line, wait, waited**	24	23	96%	64%
🔍	**Twelve words: queue, line, wait, slow, till, checkout …**	48	36	75%	100%

Precision = right hits divided by hits found. Recall = right hits divided by all 36 queue texts. A search for "crowded cashier" returns 6 of 10 texts about staff and only 4 about queues.

An embedding is a vector of numbers placed so that similar meanings sit close together



An embedding turns a message into a vector. Messages with similar meaning get nearby vectors.

Illustrative: the positions are made up.

A real embedding has hundreds of numbers: 384 for all-MiniLM-L6-v2 (model card).

You compute real ones in Colab.

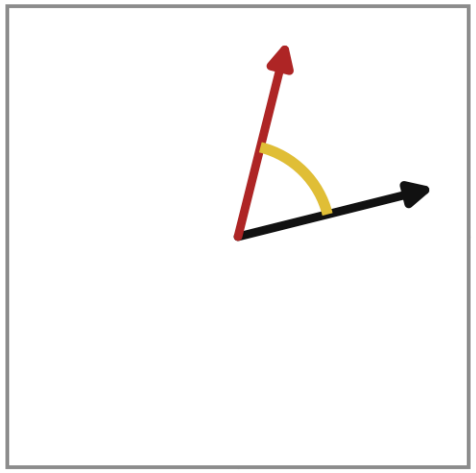
Cosine similarity measures the angle between two vectors, so length does not matter



similar meaning

angle 4 degrees

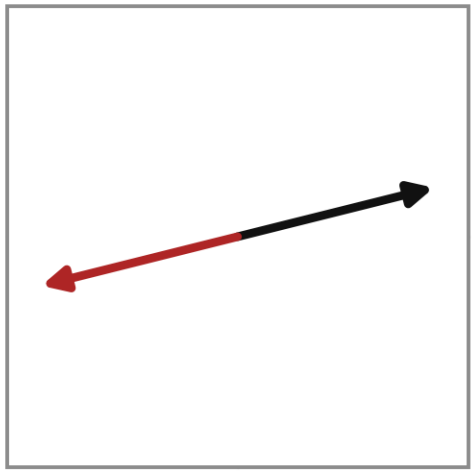
cosine 0.997



different topic

angle 62 degrees

cosine 0.471



opposite

angle 180 degrees

cosine -1.000

Cosine similarity runs from -1 to 1. Close to 1 means a very similar direction.

1 means the same direction: very similar.

0 means a right angle: nothing in common.

-1 means opposite directions.

Cosine similarity is the dot product divided by the two vector lengths

Cosine similarity

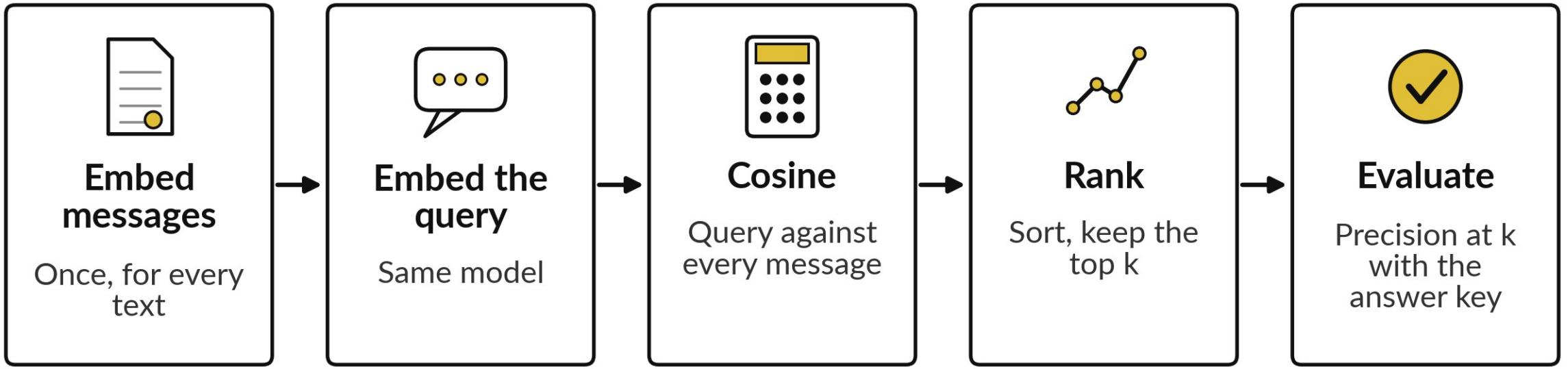
$$\cos(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|}$$

Illustrative: a = (3, 4), b = (4, 3)

$$\frac{3 \times 4 + 4 \times 3}{5 \times 5} = \frac{24}{25} = 0.96$$

Symbol	What it means	In our data
a · b	Dot product: multiply matching numbers, then add	3 × 4 + 4 × 3 = 24
|a|	Length of vector a: the square root of the sum of squares	sqrt(9 + 16) = 5
Real TF-IDF	Two queue messages that differ only in the town	0.92
Real TF-IDF	A queue message and "No queue at all", then a mobile money message	0.24 and 0.00

Semantic search ranks messages by the cosine similarity of their embeddings to the query



The recipe is identical for TF-IDF vectors. Only the vectors change.

Messages can hold personal data: remove it with a regex before you analyse

Before (illustrative)

Call me on 0700 000 111 about my refund.

A phone number is personal data.

After the regex

Call me on PHONE_REMOVED about my refund.

Names in free text are personal data too.

My understanding: the Uganda Data Protection and Privacy Act, 2019 covers personal data. Ask your data officer. Not legal advice.

Where you will use this at work: turn free-text comments into counts, labels and a search

Your job	The text you meet	What today lets you do
Customer experience analyst at a supermarket chain, using pandas	Comment cards, SMS and WhatsApp feedback	Count messages by theme and sentiment, and hand the table to Power BI or Excel.
Programme officer at an NGO, using Excel and Python	Open-ended answers in a survey	Find the most distinctive words with TF-IDF and read the mistakes before reporting.
Support team lead at a mobile money agent network	Complaint tickets	Search past tickets by meaning to find repeats, with sentence-transformers.
Credit analyst at a microfinance institution	Free-text notes on loan files	Mask phone numbers with a regex before any text leaves the team.

Read the recipe before the lab: six steps build a bag of words and TF-IDF

```
docs = [m.split() for m in clean]
vocab = sorted({w for d in docs for w in d})
X = np.array([[d.count(w) for w in vocab]
              for d in docs])
doc_freq = (X > 0).sum(axis=0).tolist()
T = X * np.log(len(docs) / np.array(doc_freq))
shared = {w: c for w, c in zip(vocab, doc_freq)
          if c > 1}
print(X.shape, shared)
```

```
(3, 14) {'checkout': 2, 'jinja': 3}
```

In plain words

Lines 1 and 2: tokens, then the sorted vocabulary.

Lines 3 and 4: count each word in each message, the bag of words.

Line 5: doc_freq, the messages that contain each word.

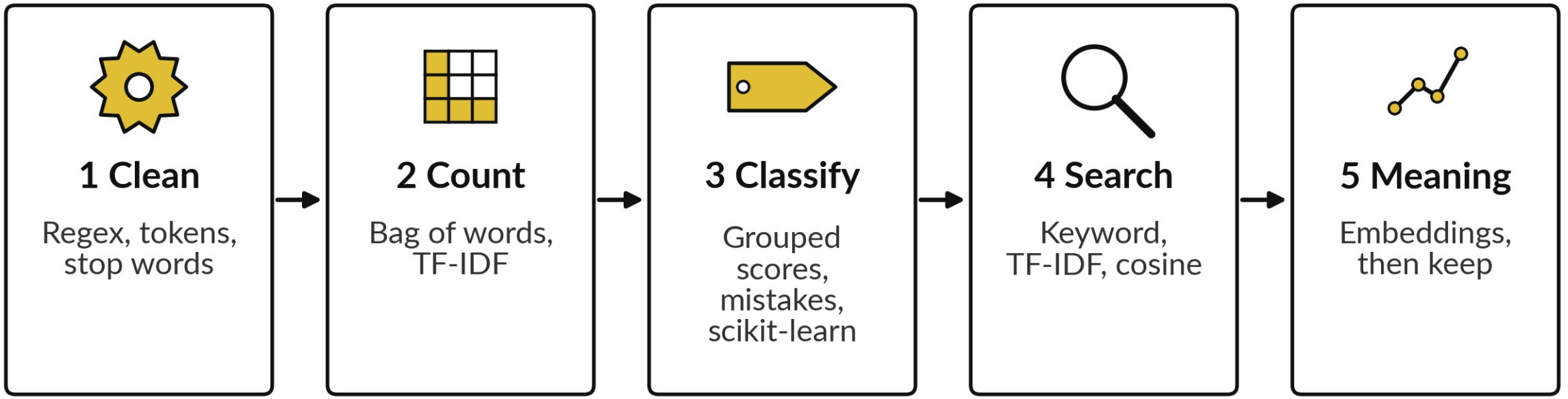
Line 6: TF-IDF. Last lines: the words in two or more messages.



Part 2: the Python lab

Close the slides. Open the notebook.

The lab has five parts, and you keep a scored file and a three-sentence finding



Five parts, twelve tasks in 45 minutes. In each one: predict, run, read the output, change one thing.

Check your numbers before you move on

What you compute	You should get
Negative messages, and the majority-class baseline share	405, which is 50.6%
Different message texts, and the vocabulary size	360 and 267
idf of queue, and its document frequency	3.74 and 19
Grouped 5-fold accuracy, and wrong answers	0.774 (0.74 to 0.81 is fine), 181 wrong
scikit-learn pipeline, grouped accuracy	about 0.79 (0.74 to 0.84 is fine)
Keyword queue: found, recall, precision	10 of 36, 27.8%, 100%
Precision at 10 for "many people waiting to pay" and for "crowded cashier"	0.7 and 0.4
Embedding matrix from all-MiniLM-L6-v2	360 rows, 384 columns

If your number differs, find the step that went wrong. Do not change a number to match.

Five likely mistakes in the lab and how to fix them

	What you see	What it means	What to do
✗	**A vocabulary size that is not 267**	The cleaning regex differs from the one in the notebook.	Copy OPEN and CLOSE exactly, and apply lower() first.
✗	**ModuleNotFoundError: sentence_transformers**	The library is not installed in this session.	Run the install cell in step 11, then run the cell again.
✗	**Accuracy of 0.94 or higher**	Copies of one text are on both sides of the split.	Make the folds with group_folds.
✗	**A cosine that is nan**	One vector has length zero (no known word in the query).	Check the query words are in the vocabulary.
✓	**Precision at 10 is lower for "crowded cashier"**	That is expected: TF-IDF only matches shared words.	Write it down as a finding. Embeddings may do better, but check.

Exit check: three questions, then compare with the short answers

	Question	Short answer
	**A classifier built on a bag of words calls "the queue was not good" positive. Why?**	It counts the words good and not and ignores their order, and good has the larger weight.
	**A keyword search finds 10 of 36 queue texts, all correct. Name both measures.**	Precision is 100 percent. Recall is 28%.
	**A cosine similarity between two message vectors is close to 1. What does it say?**	The vectors point almost the same way: the messages are very similar in the representation used.

Save your notebook and your scored file

What you keep from today

Keep your notebook and the file keep/feedback_scored.csv, with a three-sentence finding: the accuracy, one mistake you read, and one search the keyword method missed.

Our question today

How can a computer compare customer messages when it only understands numbers?

Next session

Session 22: Language models and building with them. We use these ideas to build a search that answers from your own documents.

Take away

Your lab file

A scored CSV

Your finding

3 sentences

Next session

Language models